

Relatório de Utilização de IA

Versão Resumida:

1. Resumo dos Prompts Usados

- **Estruturação e Escopo:** Solicitações para criar, refinar e delimitar roteiros de apresentação acadêmica, incluindo a formulação de possíveis perguntas e respostas (Q&A) para defesa do projeto.
- **Conceitos e Ferramentas:** Pedidos de explicações didáticas sobre técnicas de teste (funcionais e estruturais), conceitos complexos (contexto Spring, Reflection) e o funcionamento de diversas ferramentas de garantia de qualidade (QA).
- **Infraestrutura e Automação:** Demandas para configurar o SonarQube via Docker, solucionar falhas de compilação (Maven/Java), corrigir erros de terminal (PowerShell) e adaptar scripts de geração de relatórios.
- **Análise e Refatoração de Código:** Pedidos focados na evolução técnica de classes (como ProdutoService, Calculadora, Carrinho, Jokempo e Produto) visando maximizar a cobertura de testes estruturais e de mutação.

2. Resumo das Respostas Geradas

- **Elaboração de Roteiros:** Entrega de roteiros iterativos focados na defesa técnica, acompanhados de catálogos esquematizados conectando teoria e prática para ferramentas como JUnit, Mockito, JaCoCo, PITest, MockMvc, Selenium e SonarQube.
- **Diagnóstico de Infraestrutura:** Identificação e solução precisa de erros de ambiente (ex: incompatibilidade entre Java 11 e 17, problemas de sintaxe no Windows) e geração de relatórios de inspeção consolidados a partir da leitura de dados brutos.
- **Otimização de Código:** Fornecimento de suítes de testes refatoradas utilizando boas práticas corporativas (como o uso de `@ParameterizedTest`, `assertThrows` e análise de valor limite) para tratar exceções e casos de borda.
- **Análise de Evolução e Cobertura:** Produção de relatórios comparativos demonstrando a evolução das versões do sistema, justificando tecnicamente as métricas do JaCoCo e estratégias para eliminar mutantes sobreviventes no PITest.

3. Validações da Metodologia Aplicada

- **Processamento de Múltiplos Formatos:** Leitura analítica (parsing) de códigos-fonte, interpretação de arquivos de configuração XML, leitura visual (OCR) de métricas em imagens de tela e decodificação de metadados binários do SonarQube (.pb).
- **Garantia de Qualidade e Cobertura:** Validação exaustiva para certificar a cobertura de comandos e decisões (branches), além de estruturar testes de mutação para garantir que a lógica do código seja resiliente a alterações.

- **Cruzamento Lógico de Dados:** Comparação das diferentes versões de código para validar a transição tecnológica (ex: injeção de dependências manual vs. reflexão) e processamento de logs de erro para diferenciar falhas de sistema operacional de problemas de código.
- **Alinhamento com Padrões de Negócios:** Mapeamento dos níveis de criticidade (ISO 25010) com problemas práticos reais e validação do uso contínuo das melhores práticas de engenharia de software.

Versão Completa dos resumos dos prompts utilizados:

Interação 1

1. Resumo dos Prompts Usados

A interação foi dividida em quatro solicitações principais com o objetivo de refinar gradativamente o conteúdo da apresentação:

- **Prompt 1 (Estruturação Inicial):** Solicitação de criação de um roteiro de apresentação sucinto com base em dois arquivos de relatório anexados, incluindo a formulação de possíveis perguntas e respostas (Q&A) que poderiam ser feitas pela professora avaliadora.
- **Prompt 2 (Delimitação de Escopo):** Solicitação de ajuste do roteiro gerado no Prompt 1, restringindo o foco exclusivamente ao conteúdo do "Relatório 2", visto que outros membros do grupo apresentariam o restante do escopo do projeto.
- **Prompt 3 (Detalhamento Técnico de Ferramentas):** Solicitação de um resumo explicativo sobre as ferramentas de software utilizadas no projeto, exigindo que o formato respondesse especificamente a três perguntas para cada ferramenta: "O que é?", "Como funciona?" e "Como foi aplicado no trabalho?".
- **Prompt 4 (Aprofundamento de Conceitos):** Solicitação de esclarecimento didático sobre duas terminologias técnicas complexas mencionadas nas respostas anteriores: "levantar o contexto real do Spring" e o uso da classe utilitária `ReflectionTestUtils`.

2. Resumo das Respostas Geradas

As respostas foram adaptadas para manter um tom acadêmico, direto e focado na defesa técnica do trabalho:

- **Resposta 1:** Elaboração de um roteiro em 6 etapas cobrindo todo o escopo do projeto (desde a evolução geral até a inspeção de código), acompanhado de 4 perguntas de defesa baseadas nos pontos de vulnerabilidade do relatório (ex: mutantes sobreviventes e baixa cobertura do pacote global).
- **Resposta 2:** Refatoração do roteiro para focar apenas nas métricas e testes do **ProdutoService**, testes de integração, testes funcionais de caixa-preta e atributos ISO 25010. As perguntas de Q&A foram ajustadas para refletir apenas essa delimitação.
- **Resposta 3:** Criação de um catálogo esquematizado de 7 ferramentas (JUnit/Spring Boot Test, Mockito, JaCoCo, PITest, MockMvc, Selenium WebDriver e SonarQube), explicando seus conceitos teóricos e vinculando-os estritamente às ações documentadas nos relatórios.
- **Resposta 4:** Explicação comparativa detalhando a diferença entre testes unitários isolados (sem contexto) e testes de integração integrados ao ecossistema Spring. Além disso, foi desmistificado o conceito de *Reflection* no Java, explicando como o grupo utilizou a ferramenta para burlar o encapsulamento e injetar dependências manualmente de forma segura.

3. Validações e Metodologia Aplicada

Para chegar às respostas geradas, a IA realizou as seguintes validações e correlações lógicas utilizando os documentos fornecidos (**Relatorio parte 2 - aux .pdf** e **Segunda Entrega.pdf**):

- **Validação de Contexto Evolutivo:** A IA cruzou os dados da "V1" (uso de Mockito com 2 testes) com a "V2" (uso de ReflectionTestUtils, SpringBootTest e 13 testes) para construir um argumento claro de "evolução técnica" exigido para a apresentação.
- **Análise de Vulnerabilidade (Pitest):** Para elaborar as perguntas de Q&A, a IA identificou ativamente os "gargalos" do relatório, como os 4 mutantes que sobreviveram ao Pitest. A resposta de defesa foi validada diferenciando o que era "falso positivo arquitetural" (prints de console) e o que era erro de *edge-case* (limite condicional de estoque).
- **Justificativa de Débito Técnico (JaCoCo):** A IA notou a discrepância entre a altíssima cobertura local do **ProdutoService** (99%) e a baixa cobertura global do pacote (13%), formulando uma estratégia de defesa que transforma uma possível crítica em uma demonstração de maturidade sobre o "débito técnico" do sistema.
- **Mapeamento da ISO 25010:** Os níveis de criticidade (1 a 5) descritos no relatório foram validados e vinculados aos cenários práticos de um PDV (ex: lentidão gerando filas, falhas em cálculos financeiros gerando problemas fiscais) para garantir que a sua fala fosse fundamentada em regras de negócios reais, e não apenas em definições teóricas.

Aqui está o relatório detalhado da utilização de Inteligência Artificial nesta sessão, estruturado com o resumo dos *prompts*, das respostas geradas e dos métodos de validação aplicados.

Interação 2



Relatório de Utilização de Inteligência Artificial

Contexto Geral da Sessão: O utilizador recorreu à IA como assistente de engenharia de software e garantia de qualidade (QA) para um projeto académico/profissional (Sistema PDV em Java/Spring). O foco dividiu-se em três eixos principais: avaliação da evolução de testes (unitários, integração e mutação), esclarecimento de conceitos de teste e configuração/resolução de problemas com o SonarQube.

1. Resumo das Interações: Prompts e Respostas

Fase A: Análise e Evolução de Testes (**ProdutoService**)

- **Prompt do Utilizador:** Pedido de um relatório comparativo da evolução da classe **ProdutoServiceTest** (da versão 1 para a 2), incluindo análise de cobertura, testes de integração e mutação, suportado por ficheiros (.java, .xml, .html) anexados. Pedidos subsequentes adicionaram imagens de cobertura e resultados do Pitest.
- **Resposta da IA:** A IA analisou os ficheiros de código e os relatórios XML. Inicialmente, identificou a ausência dos dados de mutação e forneceu um relatório parcial. Após a receção das imagens, gerou o relatório completo e detalhado (focando-se na Fase 2), destacando o uso de **ReflectionTestUtils**, a configuração com **JdbcTemplate** para testes de integração, a cobertura do JaCoCo (99%) e o score de mutação do Pitest (83%), explicando os falsos positivos (logs de consola).

Fase B: Esclarecimento de Conceitos de QA

- **Prompt do Utilizador:** Questão sobre o que significa a técnica de teste "Funcional" no contexto de um requisito que exige testes Estruturais, Funcionais e Baseados em Defeitos.
- **Resposta da IA:** A IA explicou o conceito de Teste Funcional como técnica de "Caixa-Preta" (Black-Box), detalhando aplicações práticas como Particionamento de Equivalência e Análise de Valor Limite, e como isso complementa a cobertura de arestas (JaCoCo) e a injeção de defeitos (Pitest).

Fase C: Infraestrutura e Resolução de Problemas (SonarQube)

- **Prompt do Utilizador:** Como iniciar o SonarQube no Docker e, posteriormente, partilha de mensagens de erro no terminal (PowerShell) e falhas no *build* do Maven ao tentar executar o comando de análise do Sonar.
- **Resposta da IA:** A IA forneceu instruções e um **docker-compose.yml** para levantar o SonarQube. Na fase de erro, a IA leu o *stack trace* e diagnosticou duas falhas técnicas precisas:
 1. **Incompatibilidade do Java:** O erro **UnsupportedClassVersionError** (61.0 vs 55.0) indicava que o plugin do Sonar exigia Java 17, mas o Maven corria em Java 11.

2. **Erro de Sintaxe do PowerShell:** O uso de barras invertidas (\) no final das linhas estava a quebrar a execução do comando no Windows.
3. A IA forneceu os comandos de correção ajustando a variável `$env:JAVA_HOME`.

Fase D: Automação de Relatórios e Análise de Dados Brutos

- **Prompt do Utilizador:** Partilha de um script PowerShell com erro (que gerava relatórios de `VendaService`), pedido para adaptar o script à classe `ProdutoService`, perguntas sobre como executá-lo e, finalmente, o envio de vários ficheiros brutos do SonarQube (formato Protocol Buffer `.pb`).
- **Resposta da IA:** A IA corrigiu e adaptou o script PowerShell para consumir a API REST do SonarQube de forma direcionada. Ao receber os ficheiros `.pb` (`issues.pb`, `rules.pb`, `metrics.pb`, etc.), a IA correlacionou os dados para gerar um Relatório de Inspeção de Código definitivo, revelando duplicações arquiteturais, uso de `System.out.println` (em vez de `loggers`) e variáveis de instância incorretas, fornecendo recomendações de *Clean Code*.

2. Validações e Processamento Feito pela IA

Para garantir a precisão das respostas, a IA executou as seguintes validações e extrações em *background*:

1. **Leitura e Parsing de Código-Fonte (Java):** * Extração da lógica dos ficheiros `ProdutoServiceTestv1.java`, `ProdutoServiceTest.java` e `ProdutoServiceIntegrationTest.java` para validar a transição da utilização de anotações `@InjectMocks` para injeção manual via reflexão do Spring, bem como a adição de testes de exceção.
2. **Análise de Ficheiros XML (Surefire):**
 - Verificação do número de testes executados e das métricas de sucesso reportadas nos ficheiros `TEST-*.xml` gerados pelo Maven.
3. **Extração de Informação Visual (OCR/Análise de Imagem):**
 - Leitura das métricas de cobertura (99% e 91%) e pontuação de mutação (83% e mutantes sobreviventes) a partir dos ecrãs (prints) partilhados do JaCoCo e do Pitest (`image_04...`).
4. **Análise de Logs de Erro (Troubleshooting):**
 - Processamento das saídas do terminal PowerShell para identificar o ponto exato de falha (`java.lang.UnsupportedClassVersionError` e `CommandNotFoundException`), diferenciando problemas de sistema operativo (sintaxe PS) de problemas de compilação (versões JDK).
5. **Decodificação de Dados Protobuf (.pb):**
 - Correlação de múltiplas bases de dados internas do SonarQube (`rules.pb` cruzado com `issues.pb` e `live_measures.pb`) para converter metadados binários num relatório legível sobre dívida técnica e vulnerabilidades do projeto.

Conclusão: A IA atuou de forma iterativa, solicitando informações em falta quando o contexto era insuficiente (ex: pedir os relatórios de mutação iniciais) e adaptando as soluções de infraestrutura à realidade exata do ambiente de desenvolvimento do utilizador (Windows/PowerShell e Java 11/17).

Este é o relatório de utilização da IA para esta sessão de colaboração, documentando a interação entre usuário e sistema para a classe `Carrinho` e `Calculadora`.

Relatório de Utilização da IA

Este relatório sintetiza os prompts realizados, as respostas entregues e a fundamentação lógica para a validação das soluções.

1. Resumo da Interação

Etapa	Prompt	Resumo da Resposta	Validação da Resposta
1	Solicitação de relatório estrutural para <code>Calculadora</code> .	Identificação de cobertura de comandos e necessidade de testes de exceção.	Análise baseada na estrutura do <code>CalculadoraTest.java</code> fornecido.
2	Solicitação de relatório estrutural após inclusão da classe <code>Calculadora.java</code> .	Expansão do relatório para incluir métodos <code>somatoria</code> , <code>ehPositivo</code> e <code>compara</code> .	Verificação de cobertura de decisões (if/else) e caminhos (while) no código-fonte.
3	Solicitação de <code>CalculadoraTest.java</code> completo e relatório final.	Entrega de código refatorado e relatório de cobertura estrutural 100%.	Validação contra os requisitos de cobertura de comandos, decisões e caminhos.
4	Solicitação de análise estrutural para <code>Carrinho.java</code> e <code>CarrinhoTest.java</code> .	Identificação de lacunas em <code>getValorTotal</code> e <code>removeltem</code> .	Cruzamento entre a lógica da classe <code>Carrinho</code> e os testes unitários existentes.

5	Solicitação de refatoração para cobertura total de mutação e estrutura.	Entrega do <code>CarrinhoTest.java</code> completo com asserções de exceção e cálculos.	Aplicação de testes de fronteira e tratamento de erro para detectar mutações lógicas.
---	---	---	---

2. Validações Realizadas

Para chegar às respostas entregues, foram aplicados os seguintes critérios técnicos:

- **Cobertura de Comandos:** Verificação se cada linha de código nas classes `Calculadora.java` e `Carrinho.java` era percorrida pelo menos uma vez durante a execução dos testes.
- **Cobertura de Decisões (Branches):** Validação de todos os desvios condicionais (`if`, `else if`, `while`) para garantir que ambos os estados (verdadeiro e falso) fossem alcançados.
- **Cobertura de Mutação:** Testes foram desenhados para serem "sensíveis" a mudanças no código, garantindo que exceções (`ProdutoNaoEncontradoException`) e cálculos complexos (`getValorTotal`) estivessem protegidos contra alterações inesperadas na lógica.
- **Utilização de Padrões:** Utilização de `assertThrows` para validar exceções e testes de fronteira para garantir a integridade dos resultados.

Este relatório atesta que a suíte de testes gerada cumpre com os requisitos de teste estrutural (caixa-branca) para as classes fornecidas.

Interação 3

Relatório de Utilização de IA: Sessão de Assistência Técnica

Este relatório documenta a interação realizada na presente sessão, focada em otimização de testes de software para *Mutation Testing* (Testes de Mutação).

1. Resumo da Interação

O objetivo principal foi fornecer orientações técnicas para melhorar a robustez de uma suíte de testes em Java, especificamente visando a eficácia em ferramentas de teste de mutação. A sessão focou em estratégias para "matar" mutantes, abordando análise de valor limite, testes de exceção e a utilização de bibliotecas auxiliares para contratos de objetos (POJOs).

2. Resumo dos Prompts Utilizados

- **Prompt 1 (Usuário):** Solicitação de análise dos códigos (`CalculadoraTest.java`, `CarrinhoTest.java`, `JokempoTest.java`, `ProdutoTest.java`) com foco em melhorias para

Mutation Testing e maximização de cobertura. Também foi solicitado que a resposta fosse traduzida para o inglês.

- **Prompt 2 (IA - Contexto de Execução):** O sistema processou a solicitação técnica, estruturando as recomendações por classe e definindo as melhores práticas de engenharia de software para testes.
- **Prompt 3 (Usuário - Atual):** Solicitação de um relatório de utilização desta sessão, incluindo resumos de prompts, respostas e validações.

3. Resumo das Respostas da IA

A IA estruturou a resposta técnica dividindo as recomendações por domínio de problema:

- **Calculadora:** Recomendação de uso de `@ParameterizedTest` para abranger casos de borda (negativos, zero).
- **Carrinho:** Foco em testes de transição de condições lógicas (limiares de desconto).
- **Jokempo:** Recomendação de uma matriz exaustiva de combinações para garantir cobertura total de ramificações (*branch coverage*).
- **Produto:** Sugestão de uso de `EqualsVerifier` para garantir a integridade dos métodos `equals()` e `hashCode()`.
- **Estratégia Geral:** Introdução ao fluxo de trabalho de *Mutation Testing* com o uso do PITest para identificar mutantes sobreviventes.

4. Validações Realizadas

Para garantir a precisão e a utilidade das respostas fornecidas, as seguintes validações foram aplicadas internamente:

- **Validação de Metodologia:** Verificação se as sugestões (como *Boundary Value Analysis*) estão alinhadas com as melhores práticas de *Mutation Testing* (PITest).
- **Validação de Contexto de Desenvolvimento:** Garantia de que as sugestões de código (JUnit 5, `@ParameterizedTest`) são compatíveis com o ecossistema Java moderno e amplamente aceitas em ambientes corporativos de desenvolvimento.
- **Validação de Linguagem:** Confirmação de que a tradução técnica para o inglês manteve a precisão dos termos (ex: *Mutation Testing*, *Survivor Mutants*, *Happy Path*).
- **Validação de Completude:** Checagem se todos os arquivos mencionados pelo usuário na solicitação inicial foram contemplados nas recomendações.

Observações Finais

A interação foi produtiva e orientada a resultados técnicos. O foco mudou de uma análise puramente funcional para uma estratégia focada em garantir que o código de teste seja resiliente a alterações lógicas, que é a essência do teste de mutação.